czii-cryo-et-object-identification

2nd Place Solution

Rank	2
Team	LuoZiqian&Lion
Author	AnnieGo
Topic ID	561568
Version	Original

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561568
Retrieved with Kaggle CLI on 2026-10-21.

Acknowledgments

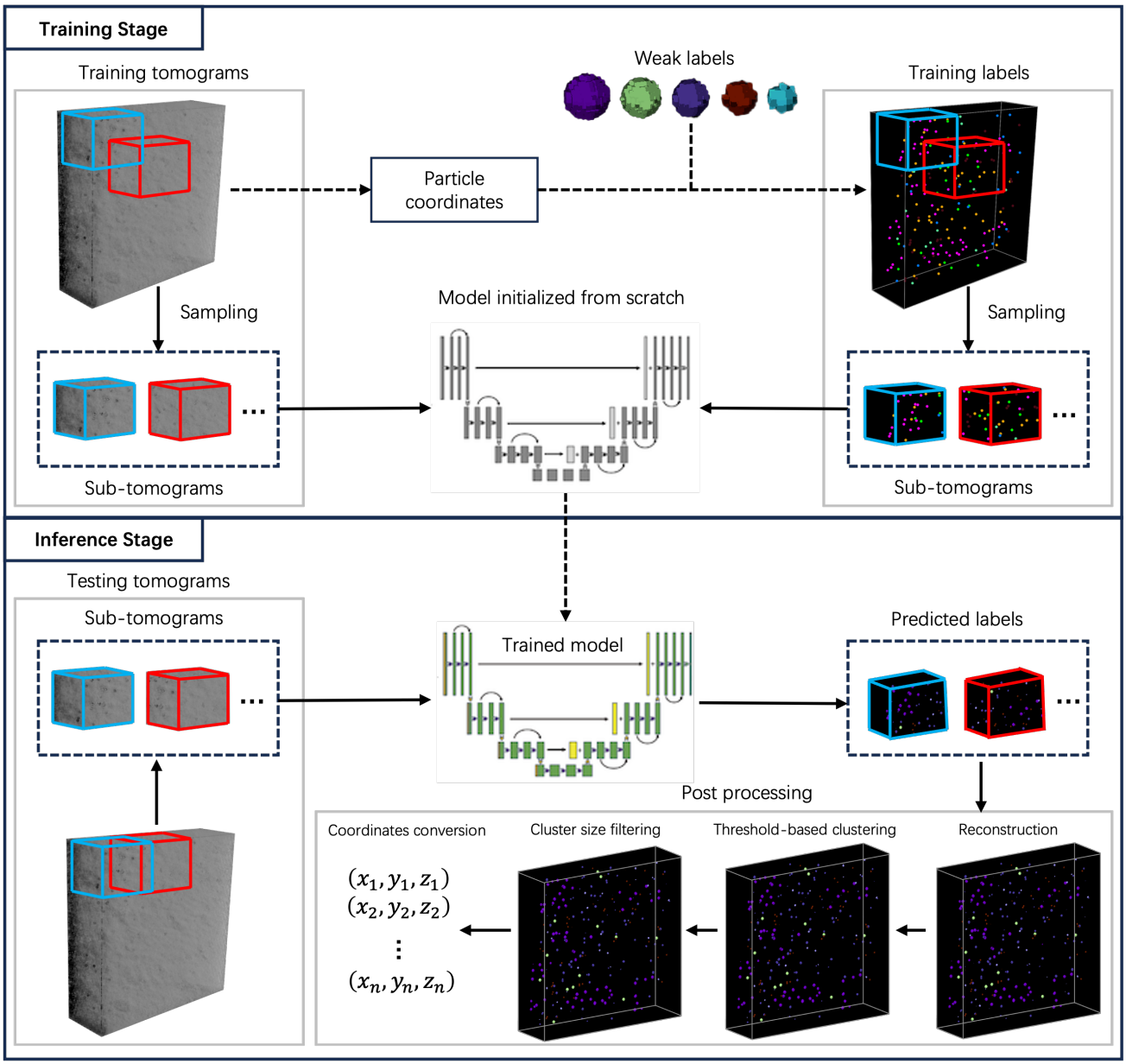
We sincerely appreciate Kaggle and the competition organizers for offering this invaluable opportunity. We also extend our gratitude to @hengck23 and @fnands for their significant contributions. **fnands's notebook** and **hengck23's discussion** provided a solid foundation for our approach. Finally, I am grateful to my teammate @luoziqian for the excellent collaboration.

Summary

Our approach is based on an ensemble of multiple lightweight segmentation models, with parameter sizes ranging from 873K to 14.2M. Following segmentation, we computed particle centroids using CC3D and filtered small clusters based on voxel count statistics.

Overall Pipeline

The overall pipeline is illustrated below:

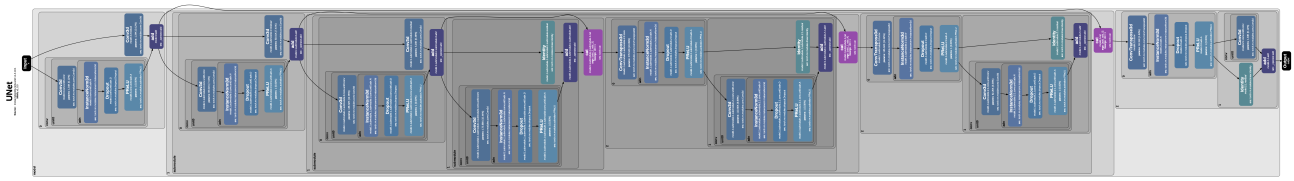


Model Architecture

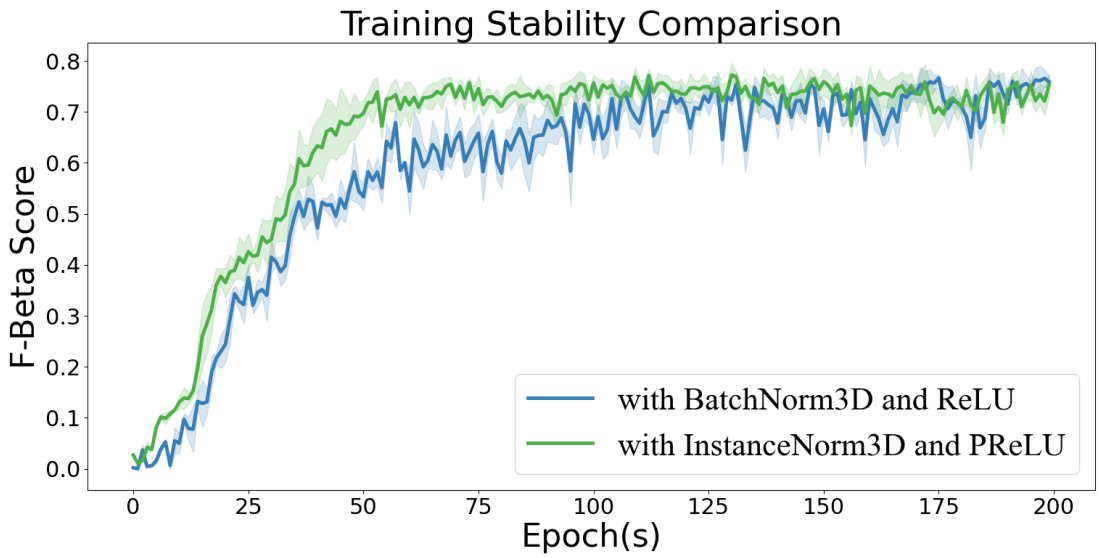
We initiated our experiments using the MONAI UNet baseline model. Throughout our trials, we observed that models with a large number of parameters were prone to overfitting and often performed worse than lighter models. Consequently, we opted for lightweight architectures such as UNet3D, VoxResNet, VoxHRNet, SegResNet, DynUNet, and DenseVNet.

However, while experimenting with VoxResNet, VoxHRNet, and DenseVNet, we encountered stability issues, including performance fluctuations and difficulties in convergence. Upon further analysis, we identified that MONAI UNet employs `InstanceNorm3d` and `PReLU`. By modifying the normalization and activation layers accordingly, we achieved more stable model performance.

The model architecture of MONAI UNet is shown below:



The following figure compares the training performance of MONAI UNet with InstanceNorm3d and PReLU versus BatchNorm3d and ReLU across 5 experiments, demonstrating that the use of InstanceNorm3d and PReLU results in more stable training.



For the final ensemble, we selected models based on their public leaderboard scores:

- UNet3D
- VoxResNet
- VoxHRNet
- SegResNet
- DenseVNet
- UNet2E3D

Training Strategy

Our training configuration was designed to ensure stability and optimal performance. We found that the **segmentation mask radius**, **loss function** and **data augmentation strategies** played a crucial role in achieving reliable results.

Segmentation Mask:

We applied ground truth masks with a customized radius for each particle.

Particle Types	Default Radius	Luoziqian's Radius	Lion's Radius
apo-ferritin	60	60 x 0.5	80 x 0.4

Particle Types	Default Radius	Luoziqian's Radius	Lion's Radius
beta-galactosidase	90	90 x 0.5	90 x 0.4
ribosome	150	150 x 0.5	150 x 0.4
thyroglobulin	130	130 x 0.5	120 x 0.4
virus-like-particle	135	135 x 0.5	150 x 0.4

Training Settings:

Lion's models:

- patch size `[128, 256, 256]` or `[128, 384, 384]`
- `200` epochs
- 7 classes
- Optimizer: AdamW with a learning rate of `0.001` (no learning rate scheduler)
- Batch size: `2` or `4`
- Drop path rate: `0.1` or `0.3`
- Model selection based on the F-beta metric, save top-5 best model
- Loss functions:
  - Tversky Loss
  - Cross-Entropy Loss with weights `[1.0, 1.0, 0.0, 2.0, 1.0, 2.0, 1.0]`

Luoziqian's models

- patch size `[128, 200, 200]` or `[128, 256, 256]`
- `100` or `300` epochs
- 6 classes
- Early stopping with `patience=20`
- Batch size: `1` or `2`
- Model selection is based on the evaluation loss. The last 100 models are saved and their F-beta scores are verified
- Loss functions:
  - Dice Loss
  - Tversky Loss
  - Cross-Entropy Loss, scaled to 0.05 or 0.1

Augmentations:

Lion's models:

- RandCropByLabelClassesd with ratios `[1, 1, 1, 1, 2, 1, 2]` for balanced sampling
- RandRotate90d
- RandFlipd
- RandAffined

- RandGaussianNoised

Luoziqian's models

- RandCropByLabelClassesd
- RandRotate90d
- RandFlipd
- RandShiftIntensityd

Model Performance with 7 TTA Summary (Partial Selection)

No	Model	Developer	Architecture	Parameters	Valid ID	Normalization	Activation	Public LB	Private LB
1	epoch122-step2952-valid_loss0.3625-val_metric0.8367.ckpt	Lion	UNet3D	1.1M	TS_86_3	InstanceNorm3d	PReLU	0.77379	0.76582
2	epoch148-step3576-valid_loss1.1154-val_metric0.7722.ckpt	Lion	UNet3D	1.1M	TS_6_4	InstanceNorm3d	PReLU	0.77021	0.76725
3	epoch153-step3696-valid_loss0.3021-val_metric0.8900.ckpt	Lion	UNet3D	1.6M	TS_69_2	InstanceNorm3d	PReLU	0.77205	0.76676
4	epoch194-step4680-valid_loss1.0213-val_metric0.8788.ckpt	Lion	UNet3D	1.1M	TS_69_2	InstanceNorm3d	PReLU	0.77390	0.76737
5	epoch138-step3336-valid_loss0.3690-val_metric0.8476.ckpt	Lion	UNet3D	1.1M	TS_73_6	InstanceNorm3d	PReLU	0.76543	0.76025
6	epoch152-step3672-valid_loss0.4333-val_metric0.7929.ckpt	Lion	DenseVNet	873K	TS_6_6	InstanceNorm3d	PReLU	0.76528	0.75417
7	epoch195-step4704-valid_loss0.4258-val_metric0.7914.ckpt	Lion	VoxResNet	7.0M	TS_6_6	InstanceNorm3d	PReLU	0.77457	0.76593
8	epoch188-step4536-valid_loss0.4231-val_metric0.8659.ckpt	Lion	VoxHRNet	1.4M	TS_73_6	InstanceNorm3d	PReLU	0.76738	0.75995
9	epoch198-step4776-valid_loss0.3471-val_metric0.8730.ckpt	Lion	VoxHRNet	1.4M	TS_73_6	InstanceNorm3d	PReLU	0.76135	0.75848
10	epoch133-val_loss0.52-val_metric0.56-step3216.ckpt	Luoziqian	UNet3D	1.1M	TS_6_4	BatchNorm3d	PReLU	0.76844	0.76320
11	epoch314-val_loss0.54-val_metric0.54-step7560.ckpt	Luoziqian	SegResNet	1.2M	TS_6_4	GroupNorm	ReLU	0.75521	0.74647

No	Model	Developer	Architecture	Parameters	Valid ID	Normalization	Activation	Public LB	Private LB
12	epoch114-val_loss0.55-val_metric0.53-Step2760.ckpt	Luoziqian	UNet2E3D	14.2M	TS_6_4	BatchNorm3d	ReLU	0.73758	0.72966

Ensemble Strategy (Partial Selection)

We select the models with the best public leaderboard scores for the final submission.

Models	Ensemble Strategy	Certainty Threshold	Public LB	Private LB	Selected
[1, 2, 3, 4, 7, 8, 12]	Average	0.23	0.79094	0.78641	❌
[1, 2, 3, 4, 7, 8, 12]	Average	0.18	0.79213	0.78630	❌
[1, 2, 3, 4, 7, 8, 12]	Average	0.15	0.79307	0.78457	❌
[1, 2, 3, 4, 7, 8, 12]	weighted	0.15	0.79247	0.78417	❌
[1, 2, 3, 4, 6, 12]	Average	0.15	0.78701	0.78389	❌
[1, 2, 3, 4, 5, 6, 7]	Average	0.15	0.79104	0.78277	❌
[1, 2, 3, 4, 7, 9, 12]	Average	0.15	0.79391	0.78283	✅
[1, 2, 3, 4, 6, 7, 9, 10, 11, 12]	Average	0.15	0.79355	0.78381	✅

Post-processing (Cluster Removal)

We used `CC3D` to convert the segmentation results into particle clusters and applied the following strategies for cluster selection:

- Threshold-based clustering (the threshold is set to 0.15 for all classses to improve recall)
- Cluster size filtering

Methods that Did Not Yield Improvements:

- Synthetic data
- Second-stage classification
- Heavy-weight models (transformer-based models)

Code

Inference Notebook
Lion's Training Code
Luoziqian's Training Code

Reference

Gubins, Ilja, et al. "SHREC 2020: Classification in cryo-electron tomograms." Computers & Graphics 91 (2020): 279-289.

Supplemental Q&A

> **FatLiuyun · 2025-02-08T06:52:59.977000**
>
> Congratulations! thank you for sharing your solution
>
> I have a question :why experiment no.2 and no.10/11/12 have such a huge different valid score(0.7x vs 0.5x,but lb very close) ?

> **AnnieGo · 2025-02-08T07:30:02.710000**
>
> Could you please clarify what is meant by “0.7x vs. 0.5x, but lb very close”? Please note that the experiments presented thus far represent only a subset of the complete experimental results. Additionally, my colleague, Luoziqian, has developed several models based on the MONAI UNet architecture that achieve a public leaderboard score exceeding 0.760; however, these models do not exhibit improved performance when ensembled.

> **FatLiuyun · 2025-02-08T07:42:32.960000**
>
> 噢是这样的，我看你实验的model列里面有val_metric这一项，no.2和no.10,no.11,no.12都是用TS_6_4作为验证集，好像这几个的验证集指标差异挺大的，想问问看是不是有什么特殊处理我错过了

> **AnnieGo · 2025-02-08T08:01:55.547000**
>
> Our experiments indicated that several factors significantly affect performance, including spatial size, mask radius, data augmentation strategies, loss functions, normalization layers, and activation layers.
>
> In the comparison between Experiment `No.2` and Experiment `No.10`, the following differences were observed matter:
>
> - Patch spatial size
> - Mask radius
> - Data augmentation strategies
> - Loss functions
> - Number of classes
>
> Furthermore, when comparing Experiment `No.2` with Experiments `No.11` and `No.12`, we found that the MONAI UNet outperformed both SegResNet and UNet2E3D. This improved performance can be attributed to several architectural features of the MONAI UNet:
>
> - The use of `InstanceNorm3d` in conjunction with `PReLU activation`.
> - An encoder downsampling factor of `1/4` as opposed to `1/8`, `1/16`, or `1/32`.
> - The implementation of a deconvolution layer using `ConvTransposed3D` instead of conventional upsampling.

- The incorporation of `residual blocks`, which enhances parameter efficiency and reduce t the likelihood of overfitting.

For both SegResNet and UNet2E3D, modifications were implemented based on our experimental investigations to facilitate training convergence. Specifically, we:

- Revised the normalization and activation layers.
- Replaced the decoder with a deconvolution layer.
- Reduced the number of decoder channels to create a more lightweight model.

Despite these modifications, neither SegResNet nor UNet2E3D outperformed the MONAI UNet.

ForcewithMe · 2025-02-09T02:21:27.833000

Congratulations! @sjtuwangshuo .

Which sample do you use to validate?

AnnieGo · 2025-02-09T04:05:20.560000

Initial Strategy:

- Split tomograms into seven folds (e.g., TS_5_4, TS_6_4, TS_69_2, etc.).
- Train each fold and save the models with the top five highest F-beta scores.
- Evaluate each of the top five models using 7× Test-Time Augmentation (TTA) on the same validation sample and compute the F-beta score.
- Select the model with the highest F-beta score after 7× TTA for submission.

Refinement:
Subsequently, `Luoziqian` observed that models validated on ['TS_5_4', 'TS_6_4', 'TS_69_2'] tend to achieve better public leaderboard performance, though not consistently. Based on this observation, I evaluate model performance using 7× TTA on these specific folds and submit the model with the highest F-beta score.

For details on the validation samples of the selected models used in the final ensemble, please refer to the `"Model Performance with 7× TTA Summary (Partial Selection)"` section of this post.

Aurora_blue · 2025-02-11T03:15:04.657000

Congratulations !
I've realized that there are a lot of room for improvement even if segmentation + cc3d approach and lack of my skills to design experiments.
In Luoziqian's Training Code, I cannot find output comp metrics in training code. Did you choose best model by other way ?

LuoZiqian · 2025-02-11T03:47:15.863000

You can change the path_dir in compute-cv-7TTA.ipynb and run it to calculate the F-beta4 score of ckpts after training. I think the Tversky Loss for each class is enough for me to initially rule out less successful experiments. Since measuring F-beta4 during training would significantly increase the experiment time and my limited computing resources, I did not calculate it during training.